

4. Tổ hợp

Tổ hợp là phần kiến thức dùng để **đếm số cách** xảy ra của một sự kiện nào đó.

4.1. Quy tắc cộng

Nếu một công việc có thể làm theo nhiều trường hợp **không giao nhau**, thì tổng số cách làm bằng tổng số cách của từng trường hợp.

Ví dụ:

Có 3 cách đi bằng xe bus.

Có 2 cách đi bằng xe máy.

Nếu chỉ được chọn 1 trong 2 loại phương tiện,
số cách đi là:

$$3 + 2 = 5$$

Nói ngắn gọn:

$$\text{số cách} = a + b$$

Quy tắc cộng thường dùng khi bài toán được chia thành nhiều trường hợp riêng biệt.

4.2. Quy tắc nhân

Nếu một công việc gồm nhiều bước liên tiếp, và mỗi bước có một số cách chọn nhất định, thì tổng số cách làm bằng tích số cách của các bước.

Ví dụ:

Có 3 cách chọn áo.

Có 2 cách chọn quần.

Mỗi bộ đồ gồm 1 áo và 1 quần.

Số cách chọn là:

$$3 * 2 = 6$$

Nói ngắn gọn:

$$\text{số cách} = a \times b$$

Quy tắc nhân thường dùng khi ta cần chọn nhiều thứ cùng lúc.

4.3. Giai thừa

Giai thừa của n , kí hiệu là $n!$, được định nghĩa:

$$n! = 1 \times 2 \times 3 \times \cdots \times n$$

Quy ước:

$$0! = 1$$

Ví dụ:

$$5! = 1 * 2 * 3 * 4 * 5 = 120$$

Giai thừa thường dùng để đếm số cách sắp xếp.

4.4. Hoán vị

Hoán vị là số cách sắp xếp n phần tử khác nhau thành một hàng.

Cái này các bạn có thể tưởng tượng đơn giản là có n vị trí. Mình sẽ bắt đầu điền số vào từng vị trí. Vị trí đầu tiên sẽ có n cách điền, tuy nhiên vị trí thứ 2 chỉ có $n - 1$ do đã mất 1 thằng ở vị trí đầu, vị trí 3 chỉ còn $n - 2$ cách do mất 2 thằng ở 2 vị trí đầu. Cứ tiếp tục xét như thế chúng ta sẽ có công thức ở dưới.

Công thức:

$$P_n = n!$$

Ví dụ:

Có 3 bạn A, B, C.
Số cách xếp 3 bạn vào một hàng là:

$$3! = 6$$

Các cách là:

ABC
ACB
BAC
BCA
CAB
CBA

4.5. Chỉnh hợp

Chỉnh hợp chập k của n phần tử là số cách chọn k phần tử từ n phần tử, **có xét thứ tự**.

Công thức:

$$A_n^k = \frac{n!}{(n-k)!}$$

Ví dụ:

Có 5 học sinh.

Chọn 2 bạn làm lớp trưởng và lớp phó.

Vì lớp trưởng và lớp phó là 2 vai trò khác nhau, nên thứ tự có quan trọng.

Số cách là:

$$A(5, 2) = 5 * 4 = 20$$

Nói đơn giản:

Chọn vị trí thứ nhất: n cách

Chọn vị trí thứ hai: $n - 1$ cách

...

Chọn vị trí thứ k : $n - k + 1$ cách

Vậy tổng số cách của mình là tích từ n đến $n - k + 1$, ứng với công thức ở trên là lấy tích từ $1 \rightarrow n$ là $n!$ rồi bỏ bớt các số từ $1 \rightarrow (n - k)$ bằng cách chia cho $(n - k)!$

4.6. Tổ hợp

Tổ hợp chập k của n phần tử là số cách chọn k phần tử từ n phần tử, **không xét thứ tự**.

Bây giờ nếu mà không xét thứ tự thì với một cách chọn ở bên tổ hợp mình sẽ đề ra được $k!$ cách ở bên chỉnh hợp (đơn giản là sắp xếp nó lại thôi chính là hoán vị). Từ đó ta chỉ cần lấy chỉnh hợp chia $k!$ là xong.

Công thức:

$$C_n^k = \binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Ví dụ:

Có 5 học sinh.

Chọn 2 bạn đi thi.

Vì chỉ cần biết 2 bạn nào được chọn, không quan trọng thứ tự chọn trước sau.

Số cách là:

$$C(5, 2) = 10$$

4.7. Phân biệt chỉnh hợp và tổ hợp

Điểm khác nhau quan trọng nhất là:

Chỉnh hợp: có xét thứ tự.
Tổ hợp: không xét thứ tự.

Ví dụ với 2 bạn A và B:

Chỉnh hợp:
AB và BA là 2 cách khác nhau.

Tổ hợp:
AB và BA chỉ là 1 cách.

Vì vậy khi đọc đề, cần để ý xem **thứ tự có quan trọng hay không**.

4.8. Một số tính chất của tổ hợp

Tính chất đối xứng

$$C_n^k = C_n^{n-k}$$

Vì chọn k phần tử cũng tương đương với việc bỏ đi $n - k$ phần tử.

Ví dụ:

$$C(5, 2) = C(5, 3)$$

Công thức Pascal

$$C_n^k = C_{n-1}^{k-1} + C_{n-1}^k$$

Ý nghĩa:

Để chọn k phần tử từ n phần tử, xét riêng phần tử thứ n :

Nếu chọn phần tử thứ n :

Ta cần chọn thêm $k - 1$ phần tử từ $n - 1$ phần tử còn lại.

Nếu không chọn phần tử thứ n :

Ta cần chọn k phần tử từ $n - 1$ phần tử còn lại.

Vì vậy:

$$C_n^k = C_{n-1}^{k-1} + C_{n-1}^k$$

4.9. Tính tổ hợp bằng DP

Dựa vào công thức Pascal, ta có thể tính trước bảng tổ hợp.

Gọi:

$$C[i][j]$$

là số cách chọn j phần tử từ i phần tử.

Khi đó:

$$C[i][j] = C[i-1][j-1] + C[i-1][j]$$

Điều kiện cơ sở:

$$C[i][0] = C[i][i] = 1$$

Code:

```
const int N = 1000;
const int mod = 1e9 + 7;

int C[N + 5][N + 5];

void build() {
    for(int i = 0; i <= N; i++) {
        C[i][0] = C[i][i] = 1;

        for(int j = 1; j < i; j++) {
            C[i][j] = (C[i-1][j-1] + C[i-1][j]) % mod;
        }
    }
}
```

Độ phức tạp:

$$O(n^2)$$

Cách này phù hợp khi n không quá lớn, ví dụ $n \leq 5000$.

4.10. Tính tổ hợp bằng giai thừa

Khi n lớn và modulo là số nguyên tố, ta thường tính:

$$C_n^k = \frac{n!}{k!(n-k)!}$$

Nhưng trong modulo, ta không chia trực tiếp được.

Thay vào đó ta dùng nghịch đảo modulo:

$$C_n^k = fact[n] \times invFact[k] \times invFact[n-k]$$

Trong đó:

```
fact[i]    = i!  
invFact[i] = nghịch đảo modulo của i!
```

Code:

```
const int N = 1000000;  
const int mod = 1e9 + 7;  
  
int fact[N + 5], invFact[N + 5];  
  
int Pow(int a, int b) {  
    int res = 1;  
  
    while(b > 0) {  
        if(b & 1) res = res * a % mod;  
        a = a * a % mod;  
        b >>= 1;  
    }  
  
    return res;  
}  
  
void build() {  
    fact[0] = 1;  
  
    for(int i = 1; i <= N; i++) {  
        fact[i] = fact[i - 1] * i % mod;  
    }  
  
    invFact[N] = Pow(fact[N], mod - 2);  
  
    for(int i = N - 1; i >= 0; i--) {  
        invFact[i] = invFact[i + 1] * (i + 1) % mod;  
    }  
}
```

```

    }
}

int C(int n, int k) {
    if(k < 0 || k > n) return 0;

    return fact[n] * invFact[k] % mod * invFact[n - k] % mod;
}

```

Độ phức tạp tiền xử lý:

$$O(n + \log MOD)$$

Mỗi lần truy vấn:

$$O(1)$$

Nhớ kỹ đọc code trên lưu ý cách tính $invFact[i]$ nhanh nhé chứ không phải mỗi lần mình đều phải lấy nó đem mũ với $mod - 2$ mà chỉ cần một lần duy nhất. Thật ra bản chất phép nhân cho $(i + 1)$ chính là bỏ bớt thừa $(i + 1)$ ở dưới mẫu số mà thôi. Giờ mình đang có $\frac{1}{5!}$ mà muốn tính $\frac{1}{4!}$ thì đơn giản chỉ cần nhân cho 5 thì số 5 dưới mẫu sẽ biến mất và đưa ra số cần tìm.

4.11. Bài toán chia kẹo cơ bản

Ta xét bài toán sau:

Có n viên kẹo giống nhau.
 Chia cho k bạn phân biệt.
 Mỗi bạn phải nhận ít nhất 1 viên.

 Hỏi có bao nhiêu cách chia?

Ví dụ có 7 viên kẹo chia cho 3 bạn, mỗi bạn ít nhất 1 viên.

Ta có thể xếp 7 viên kẹo thành một hàng:

* * * * *

Giữa 7 viên kẹo sẽ có 6 cái khe:

* _ * _ * _ * _ * _ *

Nếu muốn chia thành 3 phần cho 3 bạn, ta chỉ cần chọn 2 cái khe để đặt vạch ngăn.

Ví dụ:

* * | * * * | * *

nghĩa là:

Bạn 1 nhận 2 viên
 Bạn 2 nhận 3 viên
 Bạn 3 nhận 2 viên

Vậy bài toán trở thành:

Có $n - 1$ cái khe.
 Chọn $k - 1$ cái khe để đặt vạch ngăn.

Do đó số cách chia là:

$$C_{n-1}^{k-1}$$

Nói cách khác, số nghiệm nguyên dương của phương trình:

$$x_1 + x_2 + \dots + x_k = n$$

với:

$$x_i \geq 1$$

là:

$$C_{n-1}^{k-1}$$

với x_i là số kẹo mà bạn thứ i nhận được.

4.12. Chia kẹo có thể rỗng

Bây giờ ta xét bài toán khó hơn một chút:

Có n viên kẹo giống nhau.
 Chia cho k bạn phân biệt.
 Mỗi bạn có thể nhận 0 hoặc nhiều viên.

Hỏi có bao nhiêu cách chia?

Thật ra với bài này mình có thể đặt $a_i = x_i + 1$ thì mình sẽ nhận được bài toán $a_1 + a_2 + \dots + a_k = n + k$ với $a_i \geq 1$ chính là bài toán chia kẹo cơ bản ở trên. Nên đáp án sẽ là:

$$C_{n+k-1}^{k-1}$$

Hoặc:

Lúc này nếu dùng cách đặt vạch vào khe như trên thì sẽ bị thiếu trường hợp, vì có thể có bạn không nhận viên nào.

Ví dụ:

| * * * | * * * *

nghĩa là bạn đầu tiên nhận 0 viên.

Để xử lý trường hợp này, ta làm một mẹo nhỏ:

Cho trước mỗi bạn 1 viên kẹo giả.

Khi đó, thay vì chia n viên kẹo cho k bạn có thể nhận 0 viên, ta tưởng tượng đang chia:

$$n + k$$

viên kẹo cho k bạn, mỗi bạn ít nhất 1 viên.

Sau khi chia xong, ta lấy lại mỗi bạn 1 viên kẹo giả.

Vậy số cách chia là:

$$C_{(n+k)-1}^{k-1}$$

hay:

$$C_{n+k-1}^{k-1}$$

Nói cách khác, số nghiệm nguyên không âm của phương trình:

$$x_1 + x_2 + \cdots + x_k = n$$

với:

$$x_i \geq 0$$

là:

$$C_{n+k-1}^{k-1}$$

Ví dụ:

Có 7 viên kẹo giống nhau.
Chia cho 3 bạn.
Mỗi bạn có thể nhận 0 hoặc nhiều viên.

Số cách chia là:

$$C_{7+3-1}^{3-1} = C_9^2$$

4.13. Tổ hợp lặp

Tổ hợp lặp là bài toán chọn k phần tử từ n loại phần tử, trong đó mỗi loại có thể được chọn nhiều lần và thứ tự chọn không quan trọng.

Ví dụ:

Có 3 loại kẹo: A, B, C.
Cần chọn 4 viên kẹo.
Mỗi loại có vô hạn viên.

Một cách chọn có thể là:

A A B C

Bản chất cách chọn này chỉ cần biết:

Loại A chọn 2 viên.
Loại B chọn 1 viên.
Loại C chọn 1 viên.

Tức là ta đang cần tìm số bộ:

$$(x_1, x_2, x_3)$$

sao cho:

$$x_1 + x_2 + x_3 = 4$$

với:

$$x_i \geq 0$$

Trong đó x_i là số lần chọn loại thứ i .

Tổng quát, chọn k phần tử từ n loại phần tử, được phép lặp, chính là bài toán tìm số nghiệm nguyên không âm của:

$$x_1 + x_2 + \cdots + x_n = k$$

Theo công thức chia kẹo có thể rằng, số nghiệm là:

$$C_{k+n-1}^{n-1}$$

Mà do tính chất đối xứng của tổ hợp:

$$C_{k+n-1}^{n-1} = C_{k+n-1}^k$$

nên công thức tổ hợp lặp thường được viết là:

$$C_{n+k-1}^k$$

Ví dụ với $n = 3, k = 4$:

$$C_{3+4-1}^4 = C_6^4$$

4.14. Hoán vị vòng tròn

Nếu xếp n phần tử khác nhau trên một vòng tròn, các cách chỉ khác nhau do xoay vòng được xem là giống nhau.

Công thức:

$$(n - 1)!$$

Ví dụ:

Xếp 4 người quanh một bàn tròn.

Nếu xếp thành hàng thì có:

$$4!$$

Nhưng do xoay vòng không tạo ra cách mới,
nên số cách là:

$$(4 - 1)! = 6$$

Mẹo nhớ:

Cố định 1 người lại, sau đó xếp $n - 1$ người còn lại.

4.15. Đếm bằng phần bù

Có nhiều bài toán nếu đếm trực tiếp sẽ rất khó.

Khi đó ta có thể đếm phần bù:

$$\text{đáp án} = \text{tổng số cách} - \text{số cách xấu}$$

Ví dụ:

Đếm số cách chọn một số từ 1 đến n sao cho không chia hết cho 2.

Thay vì đếm trực tiếp,
ta có thể lấy:

n – số lượng số chia hết cho 2.

Trong tổ hợp, phần bù thường dùng khi điều kiện cần đếm là:

- Không có phần tử nào đó.
- Ít nhất một phần tử thỏa mãn.
- Không có ai đứng đúng vị trí.
- Không có hai phần tử kề nhau.

4.16. Hoán vị có phần tử trùng nhau

Ở phần hoán vị bình thường, ta xét trường hợp tất cả phần tử đều khác nhau.

Nhưng nếu có một vài phần tử giống nhau thì khi đổi chỗ các phần tử giống nhau, ta không tạo ra cách sắp xếp mới.

Ví dụ:

Có các chữ cái: A, A, B

Nếu coi 2 chữ A là khác nhau, ta có:
3! cách sắp xếp.

Nhưng thực tế 2 chữ A giống nhau,
nên mỗi cách đã bị đếm lặp 2! lần.

Vậy số cách sắp xếp là:

$$\frac{3!}{2!}$$

Tổng quát, nếu có n phần tử, trong đó:

Loại 1 xuất hiện a_1 lần.
Loại 2 xuất hiện a_2 lần.
Loại 3 xuất hiện a_3 lần.
...

với:

$$a_1 + a_2 + \dots + a_k = n$$

thì số cách sắp xếp là:

$$\frac{n!}{a_1! a_2! \dots a_k!}$$

Ví dụ:

Có chuỗi: BANANA

Chuỗi này có:

B xuất hiện 1 lần.
A xuất hiện 3 lần.
N xuất hiện 2 lần.

Vậy số cách sắp xếp khác nhau là:

$$\frac{6!}{1!3!2!}$$

Nói ngắn gọn:

Nếu có phần tử trùng nhau,
ta lấy tổng số hoán vị rồi chia cho số cách đổi chỗ của các phần tử giống nhau.

4.17. Chọn không kể nhau

Ta xét bài toán:

Có n vị trí được đánh số từ 1 đến n .
Cần chọn k vị trí sao cho không có 2 vị trí nào kề nhau.

Ta sẽ biến bài toán này thành một bài toán quen thuộc hơn.

Giả sử ta đã chọn được k vị trí. Ta đánh dấu mỗi vị trí được chọn là X.

Vì không có 2 vị trí nào kề nhau, nên giữa 2 dấu X liên tiếp bắt buộc phải có ít nhất 1 ô trống.

Ví dụ với $k = 4$, dạng cơ bản nhất phải là:

X _ X _ X _ X

Ở đây:

X là vị trí được chọn
_ là vị trí bắt buộc không được chọn

Với k dấu X, ta cần có k - 1 ô trống bắt buộc nằm giữa chúng.

Bây giờ ta gộp mỗi dấu X, trừ dấu X đầu tiên, với một ô trống bắt buộc đứng ngay trước nó:

X _ X _ X _ X

có thể nhìn thành:

X [_ X] [_ X] [_ X]

Như vậy, thay vì nghĩ là ta chọn k vị trí không kề nhau, ta nghĩ là ta chọn k "khối":

X, [_ X], [_ X], ..., [_ X]

Tổng cộng có k khối.

Các ô trống bắt buộc đã nằm sẵn trong các khối này rồi, nên sau đó các khối có thể được đặt như những vị trí bình thường.

Do mỗi khối từ khối thứ 2 trở đi đã chứa thêm 1 ô trống bắt buộc, nên số vị trí hiệu dụng bị giảm đi:

$$k - 1$$

Vậy từ n vị trí ban đầu, ta còn:

$$n - (k - 1) = n - k + 1$$

vị trí hiệu dụng.

Bài toán trở thành:

Chọn k vị trí bất kỳ trong n - k + 1 vị trí hiệu dụng.

Do đó số cách là:

$$C_{n-k+1}^k$$

4.18. Nhị thức Newton

Nhị thức Newton là công thức khai triển lũy thừa của một tổng.

Ví dụ:

$$(a + b)^2 = a^2 + 2ab + b^2$$

$$(a + b)^3 = a^3 + 3a^2b + 3ab^2 + b^3$$

Các hệ số 1, 3, 3, 1 chính là các giá trị tổ hợp:

$$C_3^0, C_3^1, C_3^2, C_3^3$$

Tổng quát:

$$(a + b)^n = \sum_{k=0}^n C_n^k a^{n-k} b^k$$